Running and understanding the Walker experiments

An environment and Python workbook

FC Portugal / RoboCup 3D imitation-learning study

Prepared 28 October 2026. Repository-specific instructions, not a generic robot installation manual.

This guide explains what is needed to set up, run and understand the BC, mixed DAgger, GAIL, varied-slip and probing experiments. It is written for a reader who can use a computer but is still learning Linux, Conda and scientific Python.

The goal is independence: identify which interpreter is running, understand what a script changes, inspect data without corrupting it, and recognize the difference between a working pipeline and a scientifically meaningful result.

How to use this workbook

Read chapters 1-8 before launching a simulator. Chapters 9-12 explain the experiment workflow and common failures. Complete the six exercises in chapter order; five are offline. Suggested timings are learning estimates, not research runtime promises.

Page map: setup 2-8; Python and arrays 9-10; workflows and troubleshooting 11-13; exercises 14-19; answers/checklist 20; sources 21.

Use a separate learning directory and fresh output names. Do not experiment inside completed research artifacts or edit receipt hashes to bypass validation.

Code in a bash block belongs in a Linux terminal. Code in a python block belongs in a Python file or interpreter. Do not copy the shell prompt, such as (fcportugal), as part of a command.

The editable Markdown and PDF are companion files. The PDF was generated from this source using the included documentation renderer; no training or simulator run was needed to make it.

1. What runs where?

Three meanings of environment

An operating environment is Linux plus system tools. A Python environment is an interpreter and its packages. The simulated environment is the physics world, robot and surface configuration. Changing one does not automatically change the others.

- Linux runs bash, compilers, the simulator process and network sockets.
- Conda isolates Python and packages such as NumPy and PyTorch.
- SimSpark/rcssserver3d advances the robot physics and sends observations.
- The Python agent chooses actions and sends them back to the server.
- RoboViz displays the simulation. It is not the physics server or the learner.

The control path

Simulator observations -> Python policy -> 16 gait actions -> gait generator and inverse kinematics -> joint commands -> simulator.

The policy does not directly output 22 independent joint velocities. The shared walking machinery interprets 16 learned gait offsets; mechanical evaluation observes 22 joint angles. Input-conditioned models receive the slip parameter as an additional input.

Offline versus live work

BC (behavioural cloning) learns to predict recorded expert actions. DAgger (dataset aggregation) additionally asks the expert to label states visited by the learner, then refits on the expanded dataset. GAIL (generative adversarial imitation learning) trains a discriminator to distinguish expert and learner samples and uses its signal to update the policy. Our GAIL recipe also uses supervised initialization and BC regularization; results describe that particular recipe.

BC fitting, saved-data probing and many report commands can run without a simulator. Expert collection, DAgger rollouts, GAIL rollouts, deployment evaluation and the viewer require one. GAIL alternates live collection and offline updates.

Managed launchers own their server: they start it, verify the generated scene, run a client and stop it. Do not start another server first. Some older demo scripts instead expect an external server; use the current command guide rather than mixing generations.

Repository map: COMMANDS.md selects commands; PIPELINES.md describes technical contracts; SESSION_HANDOFF.md records current state; CHALLENGES.md records failure history.

2. Linux and shell knowledge

Paths and the working directory

An absolute Linux path begins with /. A relative path is interpreted from your current directory. The documented research checkout is /home/pamnt/robocup/Walker; replace it with your own checkout path. It is not a required username or location.

```
pwd
cd /home/pamnt/robocup/Walker
ls
git status --short
```

pwd shows where you are. cd changes directory. ls lists files. git status identifies local changes; do not discard them to make an experiment run. Launchers normally change into the repository themselves, but python -m ... should be called from its root.

Reading a command

```
./run_submission_controls.sh audit
python -m submission_controls.run --help
```

./ means a file in this directory. audit is a positional argument. --help is an option interpreted by the program. Use help on the documented module; some old shell pipelines do not treat --help as a harmless request.

A && B runs B only if A succeeds. > writes output to a file and can overwrite it. >> appends. Ctrl+C interrupts the foreground command. A nonzero exit code normally indicates failure; a traceback is useful diagnostic evidence, not a reason to delete results.

Windows users

Run these bash/Linux pipelines inside the configured Linux installation or WSL distribution, not directly in PowerShell. Windows paths such as C:\Users\... are not Linux paths. A Linux environment does not reuse a Windows Conda installation or Windows native binaries. WSL GUI/GPU setup needs separate validation; this guide does not certify it.

Keep large active experiments on a reliable local filesystem. Do not rely on cloud synchronization as a transaction-safe live experiment store.

3. Conda, Python and the base prompt

Why a script can work from (base)

Several launchers explicitly use the following fallback interpreter:

```
/home/pamnt/miniconda3/envs/fcportugal/bin/python3
```

That executable loads the fcportugal Python packages even if the shell prompt says (base). It does not fully activate that environment for all external programs. Conda activation also changes PATH and may apply other environment settings. [S1]

```
conda env list
conda activate fcportugal
command -v python
python -c "import sys; print(sys.executable); print(sys.version)"
python -m pip --version
```

The executable and pip location should belong to the intended environment. Prefer python -m pip over a bare pip: it binds installation to the Python you selected. Do not use sudo pip to repair a project environment.

Portable launcher override

After activating your chosen environment, set the launcher override once:

```
export PYTHON_BIN="$(python -c 'import sys; print(sys.executable)')"
./run_submission_controls.sh plan
```

The command substitution obtains the active executable; the quotes preserve a path containing spaces. Child Python workers generally use sys.executable to keep the same interpreter.

Important exception: the FC Portugal native builder resolves python3.9 by name when running under Python 3.9. Therefore, activation/PATH still matters even if the launcher uses an absolute interpreter. Check command -v python3.9 before building.

Do not replace system Python or assume (base) describes the interpreter inside every subprocess.

4. Reconstructing the software environment

Inspected reference stack, not a complete lockfile

The current machine reports Ubuntu 24.04.4, Python 3.9.25, NumPy 1.26.4, PyTorch 2.8.0, SciPy 1.13.1, Matplotlib 3.9.4 and pybind11 3.0.4. Installed Debian-package versions are SimSpark 0.3.8-1 and rcssserver3d 0.7.9-1.

These are observations on 28 October 2026, not a tested universal installation recipe. Python 3.9 is a legacy, unsupported Python series. Use it only when intentionally reproducing this historical runtime; a modernized environment needs its own validation and experiment identity. [S2]

Preferred route: preserve the functioning environment

On the source machine, export into a new, private setup directory outside historical receipts:

```
mkdir -p ~/walker-setup-notes
conda env export -n fcportugal --no-builds \
  > ~/walker-setup-notes/fcportugal-environment.yml
conda list -n fcportugal --explicit \
  > ~/walker-setup-notes/conda-explicit.txt
conda run -n fcportugal python -m pip freeze \
  > ~/walker-setup-notes/pip-freeze.txt
```

Choose a fresh directory if those filenames already exist. Review exports before sharing: they may contain private channel URLs, local paths, variables or an absolute prefix. An explicit Conda export is platform-specific and does not replace the pip inventory. [S1, S3]

On a compatible machine, use a reviewed YAML with an appropriate name and without the source machine's absolute prefix:

```
conda env create -n walker-repro -f fcportugal-environment.yml
conda activate walker-repro
python -m pip check
```

No complete dependency lockfile was found in the repository audit. Obtain the source export rather than guessing the full dependency set from the six packages above. Creating an empty Conda environment alone is not a working experiment setup.

5. System dependencies and simulator setup

Python packages are only one layer

Install SimSpark and RCSSServer3D through the project's official build/download instructions for your Linux distribution. They are not supplied by installing PyTorch. Use a consistent pair of server libraries and executable. RoboViz is a separate viewer with its own Java/graphics requirements. [S4, S5]

```
command -v rcssserver3d
command -v make
command -v g++
make --version
g++ --version
```

On Ubuntu, build-essential is a common compiler/make package. Install system prerequisites through your administrator/package manager, following the simulator's version-specific instructions. Do not blindly mix a source install under /usr/local with distribution packages under /usr.

Paths this repository expects

The managed consolidated server code uses /usr/share/rcssserver3d, including rcssserver3d.rb, naosoccersim.rb and rsg/agent/nao/soccer.rsg. The preflight checker also searches /usr/local/share, but passing that checker alone does not make the managed runner portable to that layout.

Verify these files and the executable's installation agree. If your installation differs, perform a deliberate port in a separate working copy; do not create misleading symlinks or silently change a completed experiment's configuration.

The user configuration is ~/.simspark/spark.rb. If absent, follow the simulator's first-run setup instructions, then stop that manual server before using a managed launcher.

Ports

The agent connects to localhost:3100; the monitor/viewer uses localhost:3200. Managed runners require both ports free. A server, viewer connection and agent are different processes. An unused GPU does not imply the simulator is stopped.

6. Synchronous simulation and surface settings

Two required settings

In your user SimSpark configuration:

```
$agentSyncMode = true
```

In the RCSSServer3D configuration used by this installation:

```
$enableRealTimeMode = false
```

These are Ruby assignments, not bash commands. Back up the specific files before editing; use an administrator-approved editor for system files. Do not overwrite the whole configuration with these two lines. Restart any server after configuration changes.

```
python check_simulator_config.py
ss -ltnp '( sport = :3100 or sport = :3200 )'
```

The first command checks configuration text, not whether an already-running server loaded it. The second inspects listeners; if occupied, identify and stop your own process normally. Never kill every Python process or remove a lock to force concurrent access.

Why synchronization matters

Collection, training and interpreter startup take wall-clock time. Synchronous operation coordinates simulation with agent steps, reducing unintended physics advancement during computation. A 3,000-active-step evaluation at 20 ms represents 60 simulated seconds, not necessarily 60 wall-clock seconds. Startup and fall recovery are not active task time.

What varied slip means here

The training grid is 0.025, 0.1, 0.2, 1 and 10. Evaluation samples a scalar from configured Gaussian groups, then holds it fixed for the deployment. It is not a spatially random field or per-contact resampling. Managed servers copy scenes, verify import/hash and check the installed scene is unchanged. The scalar is a simulator parameter, not a directly measured real-world friction coefficient.

7. Native extensions and GPU checks

Why an import can fail before a policy runs

The agent uses compiled modules under cpp/, including ball prediction, localization and pathfinding. Their .so binaries must match the Python ABI and system libraries. ABI means the binary-level calling convention expected by the interpreter and compiled code.

After activating the intended environment, in a fresh working copy with no active research jobs:

```
python -m pybind11 --includes
python -c "from scripts.commons.Script import Script; Script.build_cpp_modules()"
python -c "from cpp.ball_predictor import ball_predictor"
python -c "from cpp.localization import localization"
python -c "from agent.Base_Agent import Base_Agent; print('agent imports OK')"
```

The build command can write native binaries. Do not rebuild or upgrade libraries during a fingerprinted run. If incompatible binaries are incorrectly reused, inspect the builder's .c_info records and interpreter selection; do not recursively delete the cpp directory. A controlled rebuild belongs in a separate validated setup.

GPU availability

```
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
nvidia-smi
```

Run nvidia-smi only on a machine with NVIDIA tools. PyTorch CUDA availability requires a compatible driver and PyTorch build; the simulator does not become GPU-accelerated because PyTorch sees a GPU. Training can fall back to CPU, but runtime changes can affect receipt identity.

Use the official PyTorch installation selector/previous-version instructions appropriate to the intended Python and CUDA versions. Do not install the latest release blindly into the historical Python 3.9 environment. The inspected local torch metadata accepts Python >=3.9; current releases need not do so. [S6]

8. Python concepts used in these pipelines

Modules, functions and objects

A module is an importable Python file. A package groups modules. python -m submission_controls.run executes a module in its package context, which is why running its file directly can break relative imports. Functions encapsulate work; classes such as a policy or Experiment hold behaviour and state. [S2]

```
from pathlib import Path
import json

root = Path.cwd()
path = root / "experiments" / "consolidated" / "v1" / "plan.json"
if path.exists():
    plan = json.loads(path.read_text())
    print(plan["budgets"])
else:
    print("No completed parent is installed here yet.")
```

Path joins path components without manual slash concatenation. plan is a dictionary: keys map to values. A list holds an ordered sequence. A for loop iterates; an if tests a condition. Indentation defines blocks.

Exceptions and resources

An exception such as FileNotFoundError explains a failed assumption. Read the final traceback line, then the call path. Do not catch every exception and pretend success.

The with statement manages a resource lifetime: an open file, loaded NumPy archive, lock or simulator. Cleanup belongs in a context manager or finally, so errors do not leave servers running.

The launcher chain

Bash chooses Python -> Python parses arguments -> the runner loads a plan -> a worker trains/evaluates -> artifacts are checked -> a completion receipt is committed.

Changing an argument can change the experiment identity. A receipt is not just a convenient “done” flag: it protects the meaning of the saved result.

9. Arrays, neural networks and data contracts

Shapes before algorithms

NumPy stores numerical arrays; PyTorch tensors additionally support gradients and devices. A batch of N ordinary observations has shape (N, 63); input-conditioned observations have (N, 64). Actions have (N, 16). Mechanical joint records have (N, 22).

An .npz file is an archive of named arrays. JSON stores readable metadata. A .pth checkpoint stores PyTorch parameters, not an executable robot. Load only trusted checkpoint files; binary model formats can have security implications.

```
import numpy as np
from pathlib import Path

paths = sorted(Path("experiments/consolidated/v1/data/original").glob(
    "seed/*/data.npz"))
if paths:
    with np.load(paths[0], allow_pickle=False) as data:
        x, y = data["obs"], data["actions"]
        print(x.shape, y.shape, x.dtype, y.dtype)
        assert len(x) == len(y)
        assert y.shape[1] == 16
        assert np.isfinite(x).all() and np.isfinite(y).all()
```

Learning vocabulary

An example is a state/action pair. A minibatch is a subset processed together. An optimizer update changes weights. An epoch is one pass through a dataset. Reusing data for more epochs is not collecting new examples.

Training fits parameters. Validation selects checkpoints/hyperparameters. Test data estimate performance after those choices. Never choose the best model using test MSE and then report that same test score as untouched evidence.

model.eval() changes evaluation behaviour of certain layers; torch.no_grad() disables gradient recording. They serve different purposes. Our inference policies are fixed during deployment even if they were previously fine-tuned.

10. Choosing the correct experiment route

A fresh reproduction versus an existing completed study

The consolidated pipeline can create its own datasets and checkpoints. Submission controls require the completed consolidated AND scratch artifacts; they are not the first command for a code-only checkout.

For a fresh small consolidated smoke, choose a new output folder:

```
./run_training.sh run --quick \
  --output experiments/learning/consolidated_smoke_01
./run_evaluation_all.sh run --quick \
  --output experiments/learning/consolidated_smoke_01
```

This runs actual simulation. First finish the installation/import/preflight checks. A smoke tests execution and contracts, not locomotion quality. Do not reinterpret its tiny budgets as a replication of the research results.

For full research reproduction, retain the same chosen output and options:

```
./run_training.sh run --output experiments/consolidated/repro_01
./run_evaluation_all.sh run --output experiments/consolidated/repro_01
```

These are potentially expensive operations. Review plan and the source README first. The completed local parent tree currently occupies about 22 GB; that is a snapshot, not a promised storage requirement. Logs, extra seeds and later controls need additional space.

To include the scratch extension, the launcher supports --include-scratch; consult scratch_comparison/README.md before using it. General probing has its own consolidated_probing entry point. Optional GAIL-improvement and wide-slip studies are separate research questions, not prerequisites for the paper controls.

Transferred historical receipts contain absolute paths, source hashes and runtime identities. Copying files to a different username is not a verified resume. Prefer a new reproduction or an explicitly audited migration; do not edit hashes by hand.

11. Current controls and the viewer

Only when completed parent artifacts are installed

```
./run_submission_controls.sh plan
./run_submission_controls.sh audit
./run_submission_controls.sh run --quick
```

The audit is offline. The quick run uses its own output and live short-horizon baselines. A full run trains seven BC controls and performs 274 new deployments. It preserves historical results and reuses verified old controls. Do not launch the full batch merely to check whether Python imports work.

Original BC receives exactly 211,050 updates; varied-only BC receives 200 total epochs for hidden/input models and three seeds. Zero native actions test the retained gait engine. Optional early-stopping diagnostics have a separate plan. See submission_controls/README.md for exact commands and limits.

Interactive viewing

```
./run_viewer.sh --list
./run_viewer.sh
```

Choose the model number and slip. Open RoboViz separately, connect to localhost:3200, then press Enter when prompted. The launcher owns the server. original preserves the legacy scene; input-conditioned policies require numeric slip.

```
./run_viewer.sh --model varied/input/seed_42/bc --slip 1
```

Use the exact label shown by --list; unavailable models are not created automatically. The viewer lists historical and completed consolidated policies, not every arbitrary checkpoint in the repository.

The movie is a qualitative sanity check, not a replacement for paired evaluation. A policy that stands still may look stable while making no useful progress. Do not run the viewer concurrently with a managed evaluation.

Offline representation probing

With the completed consolidated evaluation artifacts available, use ./run_consolidated_probing.sh audit, then ./run_consolidated_probing.sh run. These inspect saved observations and network activations without a simulator. Ridge probes test whether selected physical variables can be predicted from features; CKA compares representation geometry. Neither proves that a decoded variable causes the policy's behaviour. See consolidated_probing/README.md for separate output paths and scratch-model options.

12. Read results and diagnose failures

Files worth understanding

plan.json states the protocol. job.json specifies one task. worker.log records progress and errors. metrics.json contains training summaries. trace.npz contains time-series observations/actions. result.json summarizes one deployment. completed.json fingerprints the artifacts and inputs. A final comparison combines verified records.

Original-scene and corrected scalar-0.1 results are distinct conditions. Sampled summaries pool the documented Gaussian rounds, not arbitrary extra tests. Training seeds, task seeds and surface-draw seeds control different randomness.

Common failure -> next check

- ModuleNotFoundError -> inspect sys.executable and python -m pip --version.

- Native .so import failure -> verify ABI, PATH and build prerequisites; use a separate controlled rebuild.

- Port occupied -> identify the listener; stop your own server normally.

- Connection refused -> inspect server startup logs and the expected port.

- Socket closed or missing startup marker -> read server.log, worker.log and failure.json; a process may have exited before the client connected.

- Inputs changed -> compare plan/source/checkpoint/runtime identities; choose a new experiment when the change is intentional.

- CUDA unavailable -> check compatible driver/build; CPU may work but is slower.

Do not delete completion receipts, falsify hashes or suppress exceptions to get a green completion message. Managed jobs preserve failed attempts. Resume is typically at job boundaries, not necessarily at the last minibatch.

What the scores mean

MSE measures per-element latent-action error. SWD compares joint-state occupancy, not ordered trajectories. Falls need an exposure denominator. Jerk is kinematic, not measured torque/energy. Normalized score 1 means parity with the oracle on that surface, even if both move slowly. Probe R-squared measures readout predictability, not causal use; random features can score highly.

Exercise 1. Find the real interpreter

Goal and preparation

Learn why a (base) prompt does not prove that a launcher uses base Python. Allow 10-15 minutes. No server or research artifacts are needed. These checks are read-only.

```
conda env list
command -v python
python -c "import sys; print(sys.executable)"
sed -n '1,12p' run_submission_controls.sh
```

Write down the terminal's Python path and the launcher's fallback path. Are they identical? What happens if the fallback path does not exist on another machine?

Activate the intended environment and repeat:

```
conda activate fcportugal
python -m pip --version
command -v python3.9
export PYTHON_BIN="$(python -c 'import sys; print(sys.executable)')"
python -m submission_controls.run --help
```

If the environment has a different name, use that name. Do not create or reinstall environments just to force these commands to match a screenshot.

Success criteria

You can identify the executable, package installer and native-builder Python. You can explain why explicitly choosing a Python binary is not the same as activating a shell environment.

Reflection

Explain what each does: PATH, PYTHON_BIN, sys.executable, python -m pip, conda activate. Which affect external programs such as make and rcssserver3d? Which are specific to our launcher?

Stretch task: use an explicit interpreter to run --help without changing the active shell environment. Check that this starts no simulator.

Exercise 2. Inspect data without changing it

Goal and preparation

Understand array shapes, finite-value checks and labelled examples. Allow 20 minutes. Create ~/walker-learning for practice scripts; do not put them in fingerprinted research source directories.

Save this as ~/walker-learning/inspect_arrays.py and run it with the intended Python:

```
import numpy as np

rng = np.random.default_rng(42)
obs = rng.normal(size=(100, 63)).astype(np.float32)
expert = rng.normal(size=(100, 16)).astype(np.float32)
prediction = expert + 0.1

assert obs.shape[0] == expert.shape[0]
assert np.isfinite(obs).all()
element_mse = np.mean((prediction - expert) ** 2)
vector_error = np.mean(np.sum((prediction - expert) ** 2, axis=1))
print(obs.shape, expert.shape)
print(element_mse, vector_error)
```

Expected values are approximately 0.01 and 0.16. Why are they different if they describe the same predictions?

Now deliberately change obs to have 99 rows. Confirm that the assertion catches the mismatch. Restore it. Add one NaN in a copy and confirm the finite-value check fails. These intentional failures are confined to synthetic data.

Apply it to a real archive

Use the read-only .npz snippet in chapter 9 if parent data are available. Print array names and shapes; do not save over the archive. Explain why inputs have 63 or 64 columns while labels have 16, and why neither implies 16 robot joints.

Success means identifying a bad data contract before training, not merely making an array load without an exception.

Exercise 3. Train a tiny regressor

Goal and preparation

Learn the difference between fitting, evaluation and selection. Allow 25-40 minutes. This synthetic CPU exercise needs PyTorch but no simulator, GPU or robot data. Save it as ~/walker-learning/tiny_fit.py.

```
import torch

torch.manual_seed(7)
x = torch.randn(256, 4)
weight = torch.tensor([[1.0], [-2.0], [0.5], [0.0]])
y = x @ weight + 0.2
model = torch.nn.Linear(4, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(100):
    model.train()
    optimizer.zero_grad()
    loss = ((model(x[:160]) - y[:160]) ** 2).mean()
    loss.backward()
    optimizer.step()

model.eval()
with torch.no_grad():
    validation = ((model(x[160:208]) - y[160:208]) ** 2).mean()
    test = ((model(x[208:]) - y[208:]) ** 2).mean()
print(float(validation), float(test))
print(model.weight.detach(), model.bias.detach())
```

The weights should approach the known mapping and both errors should be small. Exact floating-point values can vary. Explain why no gradient should be taken on the validation or test slices.

Modify the loop to record validation loss each epoch and save the best weights using copy.deepcopy(model.state_dict()). Keep test data unread until selection is finished. Then add label noise and compare the best-validation epoch with the final epoch.

This exercise is ordinary linear regression, not the project's closed-form ridge probe. The probe adds regularization and freezes the network that supplies its features. A high score on a known simple mapping tests implementation, not robotic understanding.

Exercise 4. Count the training budget

Goal and preparation

Distinguish examples, exposures, epochs, updates and environment steps. Allow 15-20 minutes. Use Python's standard library only.

```
from math import ceil

samples = 170_000
batch = 256
updates_per_epoch = ceil(samples / batch)
target_updates = 211_050
full_epochs, extra_batches = divmod(target_updates, updates_per_epoch)
exposures = full_epochs * samples + extra_batches * batch

print(updates_per_epoch)
print(full_epochs, extra_batches)
print(exposures)
print("BC at 100 epochs:", 100 * samples, "exposures")
print("BC at 200 epochs:", 200 * updates_per_epoch, "updates")
```

Expected: 665 updates/epoch; 317 full epochs plus 245 minibatches; 53,952,720 exposures. Each full epoch includes one final 16-example minibatch. Here the 245 extra minibatches are all full-size.

Questions

1. Why is 170,000 divided by 256 rounded up, not down?

2. Does training for 200 epochs collect 34 million new expert examples?

3. Why does matching updates not match DAgger's oracle-query cost?

4. Why must we record the selected checkpoint's update count separately from the allocated budget?

5. A 3,000-step run at 20 ms/step lasts how many active simulated minutes?

Stretch task: compute exposures for a final partial epoch that reaches the last, short minibatch. Why would extra_batches * batch then need correction?

Success means explaining why matched sample counts alone did not settle the BC-versus-DAgger optimization question in the consolidated study.

Exercise 5. Reproducibility and diagnostic checks

Goal and preparation

Understand checksums and deliberate failure handling. Allow 15-25 minutes. Use only synthetic files in your learning directory.

```
from pathlib import Path
import hashlib
import tempfile

with tempfile.TemporaryDirectory(prefix="walker_hash_lesson_") as folder:
    path = Path(folder) / "toy.txt"
    path.write_text("original data\n")
    before = hashlib.sha256(path.read_bytes()).hexdigest()
    path.write_text("changed data\n")
    after = hashlib.sha256(path.read_bytes()).hexdigest()
    print(before == after)
```

Expected: False. A hash detects changed bytes; it does not establish scientific correctness or identify which physical effect caused a result.

Read-only repository checks

From the repository root, run:

```
python -m unittest submission_controls.test_pipeline
python -m submission_controls.run --help
```

These tests use temporary synthetic artifacts. They check exact budgets, checkpoint selection, zero outputs, resume/tamper behaviour and reporting. They do not establish that a robot walks well. If tests fail, keep the traceback and identify the failing assumption before changing code.

Write a short response to this scenario: “The runner says inputs changed after I upgraded PyTorch. Can I edit completed.json to match?” The safe answer should distinguish a new experiment identity from a genuinely verified resume.

Do not reproduce the tampering exercise on a real checkpoint, receipt, dataset or simulator configuration.

Exercise 6. A safe smoke and a model demo

Goal and prerequisites

Connect the software layers without launching a full research batch. Allow at least 30 minutes after setup; build and simulation times vary. All chapters 1-7 checks must pass. No other server or live experiment may be using ports 3100/3200.

Choose ONE route. Route A needs a completed audited parent and scratch extension. Route B is for a code checkout that must collect new smoke data.

```
# Route A: completed local study exists
./run_submission_controls.sh audit
./run_submission_controls.sh run --quick \
  --output experiments/learning/controls_smoke_01
```

```
# Route B: create a fresh consolidated smoke
./run_training.sh run --quick \
  --output experiments/learning/consolidated_smoke_01
./run_evaluation_all.sh run --quick \
  --output experiments/learning/consolidated_smoke_01
```

Use fresh names. Repeat the same command afterward: completed jobs should be verified and skipped. Do not change flags and expect the old receipt to remain valid.

For Route A, inspect evaluation/summary/comparison.json. Confirm the short active horizon and that fresh short-horizon oracle/random baselines were used. Check worker/server logs and completion receipts. Do not interpret tiny-smoke scores as research findings.

After the smoke has fully stopped, run the viewer. For Route B, use ./run_viewer.sh --root experiments/learning/consolidated_smoke_01. Select an available model and numeric slip, connect RoboViz to localhost:3200, then follow the prompt. Smoke-trained models may walk poorly.

Write down: who started the server, which Python ran, where outputs were saved, which model/slip was selected, and why the demonstration is not a statistical evaluation.

Answer guide and readiness checklist

Exercises 1-3

The active shell's interpreter can differ from the launcher's explicit fallback. PYTHON_BIN is a project convention; PATH controls ordinary executable lookup; sys.executable reports the running interpreter. Activation affects subprocess lookup, while an absolute interpreter alone does not reproduce all activation settings.

The synthetic action error is 0.01 per element but 0.16 when summing over 16 outputs. Array rows align samples; columns represent features/actions. Detecting NaNs or mismatched rows before fitting is part of correctness.

The tiny linear model should recover approximately [1, -2, 0.5, 0] with bias 0.2. Gradient descent operates on training data. Validation chooses settings/checkpoints; test estimates the selected model afterward. Copy a state dictionary when retaining a checkpoint in memory, otherwise references can continue to change.

Exercises 4-6

An incomplete final batch still needs an update. Epochs reuse examples. DAgger also pays for learner-state collection and labels; equal update counts do not erase those costs. A selected best checkpoint can precede the final budget endpoint. 3,000 steps at 0.02 seconds gives one active simulated minute.

Changing a file changes its checksum. It does not authorize updating a historical receipt. A changed runtime normally needs a fresh planned experiment or an explicitly reviewed compatibility mechanism. A smoke validates execution/contracts; it does not prove convergence, significance or robustness.

Ready for an independent research run?

- You can identify the interpreter, package environment and system simulator.
- Imports and native modules work under the selected Python.
- Configurations, paths, ports and managed-server ownership are understood.
- You know whether the command requires existing parent artifacts.
- Output location, budget, seeds and resume behaviour are written down.
- Training, validation, test and active-time accounting are kept separate.
- A fresh smoke passed and the old study remains unchanged.

If any item is uncertain, stop at an offline plan/audit instead of launching the full batch.

Sources and further reading

Official software references

[S1] Conda: managing environments. Activation, environment creation and isolation.

https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html

[S2] Python tutorial: modules. Consult the documentation matching your interpreter; the historical 3.9 documentation also flags that version as unsupported.

https://docs.python.org/3.9/tutorial/modules.html

[S3] Conda: environment export command. Review platform-specific and private information before sharing exports.

https://docs.conda.io/projects/conda/en/stable/commands/env/export.html

[S4] SimSpark project: downloads and RCSSServer3D. Use project instructions for the selected installation/platform.

https://robocup-sim.gitlab.io/SimSpark/downloads.html

[S5] RoboViz project. Viewer installation and its Java/graphics requirements are separate from Python.

https://github.com/magmaOffenburg/RoboViz

[S6] PyTorch installation guide. Choose a compatible Python/platform/compute build; do not silently upgrade the recorded experiment runtime.

https://pytorch.org/get-started/locally/

Repository sources of truth

COMMANDS.md; PIPELINES.md; SESSION_HANDOFF.md; CHALLENGES.md; consolidated/README.md; scratch_comparison/README.md; submission_controls/README.md; consolidated_probing/README.md.

Implementation inspected: run_submission_controls.sh, check_simulator_config.py, scripts/commons/Script.py, consolidated/server.py, consolidated/worker.py, viewer/run.py and behaviors/custom/Walk/Env.py.

This guide is an onboarding aid, not a complete dependency lockfile, a certified cross-platform installation, or evidence that a new reproduction has passed. The versions listed were read from the local machine. Package installation, simulator setup and full research experiments were not performed to produce this document.